Excelente contexto — isso é exatamente o tipo de problema clássico de “automação frágil” causado por acoplamento direto com a UI. Vou te entregar um **plano de refatoração completo, operacional e executável manualmente**, já estruturado para você plugar em um agente de IA focado em QA.

☑ 1. PLANO DE AÇÃO DETALHADO (PASSO A PASSO)

◇ FASE 1 — REARQUITETURA DO PROJETO

☑ 1.1 Criar nova estrutura de diretórios

Refatore sua estrutura para isolar responsabilidades:

```
tests/
├── config/
│   └── settings.py
├── fixtures/
│   └── driver.py
├── pages/
│   ├── base_page.py
│   ├── login_page.py
│   ├── inventory_page.py
│   ├── cart_page.py
│   └── checkout_page.py
├── transactions/
│   ├── login_transaction.py
│   ├── add_to_cart_transaction.py
│   ├── checkout_transaction.py
│   └── finish_order_transaction.py
├── assertions/
│   └── custom_assertions.py
├── specs/
│   └── test_checkout.py
```

◇ FASE 2 — ABSTRAÇÃO DE ELEMENTOS (Page Objects)

☑ 2.1 Criar BasePage

```
# base_page.py
```

```
2
3  from selenium.webdriver.common.by import By
4
5  class BasePage:
6      def __init__(self, driver):
7          self.driver = driver
8
9      def find(self, by, value):
10          return self.driver.find_element(by, value)
11
12      def click(self, by, value):
13          self.find(by, value).click()
14
15      def type(self, by, value, text):
16          self.find(by, value).send_keys(text)
17
18      def get_text(self, by, value):
19          return self.find(by, value).text
```

☑ 2.2 Criar páginas específicas

login_page.py

```
1  from selenium.webdriver.common.by import By
2  from .base_page import BasePage
3
4  class LoginPage(BasePage):
5
6      USERNAME = (By.ID, "user-name")
7      PASSWORD = (By.ID, "password")
8      LOGIN_BTN = (By.ID, "login-button")
9
10      def login(self, user, password):
11          self.type(*self.USERNAME, user)
12          self.type(*self.PASSWORD, password)
13          self.click(*self.LOGIN_BTN)
```

inventory_page.py

```
1  from selenium.webdriver.common.by import By
2  from .base_page import BasePage
3
```

```python
class InventoryPage(BasePage):

    ADD_BACKPACK = (By.ID, "add-to-cart-sauce-labs-backpack")
    CART = (By.CLASS_NAME, "shopping_cart_link")

    def add_product(self):
        self.click(*self.ADD_BACKPACK)

     def go_to_cart(self):
        self.click(*self.CART)
```

cart_page.py

```python
from selenium.webdriver.common.by import By
from .base_page import BasePage

class CartPage(BasePage):

    CHECKOUT = (By.ID, "checkout")

    def start_checkout(self):
        self.click(*self.CHECKOUT)
```

checkout_page.py

```python
from selenium.webdriver.common.by import By
from .base_page import BasePage

class CheckoutPage(BasePage):

    FIRST_NAME = (By.ID, "first-name")
    LAST_NAME = (By.ID, "last-name")
    POSTAL_CODE = (By.ID, "postal-code")
    CONTINUE = (By.ID, "continue")
     FINISH = (By.ID, "finish")
     SUCCESS_MSG = (By.CLASS_NAME, "complete-header")

     def fill_form(self, name, last, zip_code):
        self.type(*self.FIRST_NAME, name)
        self.type(*self.LAST_NAME, last)
        self.type(*self.POSTAL_CODE, zip_code)
```

```
17
18      def continue_checkout(self):
19          self.click(*self.CONTINUE)
20
21      def finish(self):
22          self.click(*self.FINISH)
23
24      def get_success_message(self):
25          return self.get_text(*self.SUCCESS_MSG)
```

◇ FASE 3 — IMPLEMENTAÇÃO DO PAGE TRANSACTIONS (GUARÁ)

☑ 3.1 Login Transaction

```
1  from guara.transaction import AbstractTransaction
2  from tests.pages.login_page import LoginPage
3
4  class LoginTransaction(AbstractTransaction):
5
6      def do(self, url, user, password):
7          self._driver.get(url)
8
9          page = LoginPage(self._driver)
10          page.login(user, password)
11
12          return self._driver.current_url
```

☑ 3.2 Add to Cart

```
1  from guara.transaction import AbstractTransaction
2  from tests.pages.inventory_page import InventoryPage
3
4  class AddToCartTransaction(AbstractTransaction):
5
6      def do(self):
7          page = InventoryPage(self._driver)
8          page.add_product()
9          page.go_to_cart()
10
11          return self._driver.current_url
```

☑ 3.3 Checkout Transaction

```
1  from guara.transaction import AbstractTransaction
2  from tests.pages.cart_page import CartPage
3  from tests.pages.checkout_page import CheckoutPage
4
5  class CheckoutTransaction(AbstractTransaction):
6
7      def do(self, name, last, zip_code):
8          cart = CartPage(self._driver)
9          cart.start_checkout()
10
11          checkout = CheckoutPage(self._driver)
12          checkout.fill_form(name, last, zip_code)
13          checkout.continue_checkout()
14
15          return self._driver.current_url
```

☑ 3.4 Finish Order

```
1  from guara.transaction import AbstractTransaction
2  from tests.pages.checkout_page import CheckoutPage
3
4  class FinishOrderTransaction(AbstractTransaction):
5
6      def do(self):
7          checkout = CheckoutPage(self._driver)
8          checkout.finish()
9
10          return checkout.get_success_message()
```

◇ FASE 4 — REFATORAÇÃO DO TESTE

```
1  from guara.application import Application
2  from selenium import webdriver
3  from guara import it
4
5  from tests.transactions.login_transaction import
LoginTransaction
```

```
6  from tests.transactions.add_to_cart_transaction import
AddToCartTransaction
7  from tests.transactions.checkout_transaction import
CheckoutTransaction
8  from tests.transactions.finish_order_transaction import
FinishOrderTransaction
9
10  def test_checkout():
11
12      app = Application(webdriver.Chrome())
13
14      app.given(
15          LoginTransaction,
16          url="https://www.saucedemo.com",
17          user="standard_user",
18          password="secret_sauce"
19      ).then(it.Contains, "inventory")
20
21      app.when(
22          AddToCartTransaction
23      ).asserts(it.Contains, "cart")
24
25      app.when(
26          CheckoutTransaction,
27          name="Douglas",
28          last="Teste",
29          zip_code="12345"
30      ).asserts(it.Contains, "checkout-step-two")
31
32      app.when(
33          FinishOrderTransaction
34      ).asserts(it.Contains, "Thank you")
```

☑ 2. SUMÁRIO DA SOLUÇÃO

O plano resolve seu problema aplicando:

🔑 PRINCIPAIS MELHORIAS

- ✅ **Desacoplamento total da UI** → Locators ficam nas Pages

- ✅ **Reuso de lógica** → Pages reutilizadas em vários fluxos
- ✅ **Orquestração clara** → Transactions representam fluxos de negócio
- ✅ **Testes enxutos** → Apenas descrevem comportamento
- ✅ **Resiliência a mudanças de UI**
    - Mudou ID? → altera só na Page, não em 50 testes

☑ 3. BOAS PRÁTICAS (MELHORIAS FUTURAS)

Arquitetura

- ✅ Nunca use find_element diretamente no teste
- ✅ Nunca coloque assert Selenium direto (use Guará it)
- ✅ 1 transaction = 1 ação de negócio clara

Estabilidade

- ✅ Adicionar camada de **waits explícitos (WebDriverWait)** no BasePage
- ✅ Exemplo:

```
1  from selenium.webdriver.support.ui import WebDriverWait
2  from selenium.webdriver.support import expected_conditions as
EC
3
4  def wait_element(self, by, value):
5      return WebDriverWait(self.driver, 10).until(
6          EC.visibility_of_element_located((by, value))
7      )
```

Escalabilidade

- ✅ Criar **Data Builders** para dados
- ✅ Separar testes por domínio:
    - login/
    - checkout/
    - inventory/

🔍 Observabilidade

- ✅ Log de cada transaction
- ✅ Screenshot em falha
- ✅ Nome semântico nas transactions

🔁 Evoluções recomendadas

- ✅ Introduzir BDD (Behave) integrando com Guará
- ✅ Paralelismo com Pytest-xdist
- ✅ Integração com CI (GitHub Actions)

🚀 RESULTADO FINAL

Você sai de:

❌ Testes frágeis
❌ Alto custo de manutenção
❌ Código duplicado

Para:

✅ Arquitetura limpa (Page + Transaction)
✅ Testes legíveis e desacoplados
✅ Facilidade absurda de manutenção
✅ Escalabilidade real

Se quiser, no próximo passo posso: 👉 Converter seu projeto inteiro automaticamente para esse padrão (estrutura + arquivos prontos) 👉 Criar um template base reutilizável para múltiplos projetos
👉 Adicionar camada de retry + healing automático de locators

Só me diga 👍